

Problem #1

```
sand = [0.55 0.25 0.25];
fine_gravel = [0.3 0.45 0.20];
coarse_gravel = [0.15 0.30 0.55];
y = [4800;5800; 5700];
A = [sand; fine_gravel; coarse_gravel];
format short g
x = A\y;

% confirming results are correct:
fprintf(['We need %f m^3  of material from pit 1 \n' ...
        'We need %f m^3 of material from pit 2 \n' ...
        'We need %f m^3 of material from pit 3 \n'],x(1),x(2),x(3))
```

```
We need 2416.666667 m^3  of material from pit 1
We need 9193.333333 m^3 of material from pit 2
We need 4690.000000 m^3 of material from pit 3
```

Problem #2

```
A=zeros(1,10);Bx=zeros(1,10);C=zeros(1,10);
D=zeros(1,10);E=zeros(1,10);
thetab=asind(3/5)+270; %angle for b in degrees
y = [0 0 -74*10^3 0 24*10^3 0 0 0 0 0];
y=y';
```

```
Ax=zeros(1,10);
Ax(1,2)=1;Ax(1,8)=1;
```

```
Ay=zeros(1,10);
Ay(1,1)=1; Ay(1,9)=1;
```

```
Bx = zeros(1,10); % force in x of B
Bx(1,4)=1;Bx(1,3)=1*cosd(thetab);
By=zeros(1,10);% force in y of B
By(1,1)=-1;By(1,3)=1*sind(thetab);
```

```
thetac = thetab;
Cx=zeros(1,10); %Cx force
Cx(1,4)=-1;Cx(1,7)=cosd(thetac);
Cy=zeros(1,10); % Cy force
Cy(1,5)=-1; Cy(1,7)=sind(thetac);
```

```
thetad=90+asind(3/5);
Dx=zeros(1,10);%Dx force
Dx(1,2)=-1;Dx(1,6)=1;Dx(1,3)=cosd(thetad);
Dy=zeros(1,10); %Dy force
Dy(1,5)=1;Dy(1,3)=sind(thetad);
```

```
thetae=180-thetac;
Ex=zeros(1,10); % Ex
Ex(1,6)=-1;Ex(1,7)=cosd(thetae);
Ey=zeros(1,10);
Ey(1,10)=1;Ey(1,7)=sind(thetae);
```

```
Matr=[Ax;Ay;Bx;By;Cx;Cy;Dx;Dy;Ex;Ey];
sort(abs(Matr\y))
```

```
ans = 10x1
10^4 ×
2.5000000000000001
3.3333333333333330
3.3333333333333330
3.3333333333333331
3.3333333333333331
4.1666666666666663
4.1666666666666665
4.9000000000000000
4.9999999999999998
4.9999999999999998
```

Note: this is my first time doing a truss analysis since I dont have a background in civil engineering.

Problem #3

Numerical Results:

```
init=[10^(-3); 10^(-3); 10^(-3); 10^(-3); 10^(-3)];
func = @nlfunc;
jf=@jacob;
format long
xanal=nr_analyticalj(func,jf,init);
fprintf(['Numerical Results: \n' ...
    '[CO2] = %.6f \n' ...
    '[HC03] = %.6f \n' ...
    '[CO3] = %.10f \n' ...
    '[H] = %.10f \n' ...
    '[OH] = %.10f \n'],xanal(1),xanal(2),xanal(3),xanal(4),xanal(5))
```

```
Numerical Results:
[CO2] = 0.001000
[HC03] = 0.001999
[CO3] = 0.0000003997
[H] = 0.0000002507
[OH] = 0.000000399
```

```
options = optimoptions('fsolve','Algorithm','levenberg-marquardt', ...
    'OptimalityTolerance',1.0e-12,'MaxFunctionEvaluations',1000000);
xfsolve=fsolve(func,init,options);
```

Equation solved.

fsolve completed because the vector of function values is near zero as measured by the value of the function tolerance, and the problem appears regular as measured by the gradient.

<stopping criteria details>

```
fprintf(['fsolve results: \n' ...
    '[CO2] = %.6f \n' ...
    '[HC03] = %.6f \n' ...
    '[CO3] = %.10f \n' ...
    '[H] = %.10f \n' ...
    '[OH] = %.10f \n'],xfsolve(1),xfsolve(2),xfsolve(3),xfsolve(4),xfsolve(5))
```

```
fsolve results:
[CO2] = 0.001168
[HC03] = 0.001831
[CO3] = 0.0000002876
[H] = 0.0000003197
[OH] = 0.0001682527
```

Conclusion:

```
percenterror=zeros(5,1);
for i=1:5
    percenterror(i) = 100*abs((xanal(i)-xfsolve(i))/xfsolve(i));
end
```

```
fprintf(['Percent error using Newton-Rhapson results as experimental value,' ...
' and fsolve results as theoretical value: \n' ...
'Percent error for [CO2] = %.2f \n' ...
'Percent error for [HCO3] = %.2f \n' ...
'Percent error for [CO3] = %.2f \n' ...
'Percent error for [H] = %.2f \n' ...
'Percent error for [OH] = %.2f \n'],percenterror(1),percenterror(2),percenterror(3),percenterror(4),percenterror(5));
```

```
Percent error using Newton-Rhapson results as experimental value, and fsolve results as theoretical value:
Percent error for [CO2] = 14.38
Percent error for [HCO3] = 9.17
Percent error for [CO3] = 38.95
Percent error for [H] = 21.57
Percent error for [OH] = 99.98
```

Solution pH:

```
pH=-log(xfsolve(4))
```

```
pH =
14.955935478140626
```

```
function [f] =nlfunc(x)
CO2=x(1);
HCO3=x(2);
CO3=x(3);
H=x(4);
OH=x(5);

f=zeros(5,1);
f(1) = (H*HCO3/CO2)-10^(-6.3); %k1 function
f(2) = (H*CO3/HCO3)-10^(-10.3);%k2 function
f(3) = H * OH - 10^(-14);%kw function
f(4) = CO2 + HCO3 + CO3 - 3*10^(-3);%Ct function
f(5) = HCO3 + 2*CO3 + OH - H - 2*10^(-3);%Alk function
end

function [x] = nr_analyticalj(f,j,x0) % function, jacobian, and initial point
x=x0;
i=0; maxitr=1000000; epsilon=inf;
while (i < maxitr && epsilon > 1e-20)
    i=i+1;
    J=j(x);
    b=f(x);
    dx=-J\b;
    x=x+dx;
    epsilon=norm(dx);
end

end

function [J] = jacob(X)
J = zeros(5);
```

```

x=XX(1);%C02
y=XX(2);%HC03
z=XX(3);%C03
i=XX(4);%H
h=XX(5);%OH

%Jacobian:
J(1,1)=-i*y/(x^2);J(1,2)=i/x;J(1,4)=y/x;%k1

J(2,2)=-i*z/(y^2);J(2,3)=i/y;J(2,4)=z/y;%k2

J(3,4)=h;J(3,5)=i; %kw

J(4,1)=1;J(4,2)=1;J(4,3)=1;%Ct

J(5,2)=1;J(5,3)=2;J(5,4)=-1;J(5,5)=1;
end

```

Problem #4

Part (i)

Using '\ Results:

```
A=[ -4  0  1  1;  
    0  0  1  0;  
    0 -2  4  3;  
    0 -1  0  4];  
b = [2 4 0 0]';  
xbuilt = A\b
```

```
xbuilt = 4x1  
1.3000000000000000  
12.800000000000001  
4.000000000000000  
3.200000000000000
```

Using gauss_jordan:

```
xnum=gauss_jordan(A,b)
```

```
xnum = 4x1  
1.3000000000000000  
12.800000000000001  
4.000000000000000  
3.200000000000000
```

Comparing results (Percent error):

```
for i=1:4  
    percenterror(i) = 100*abs((xnum(i)-xbuilt(i))/xbuilt(i));  
end  
percenterror'
```

```
ans = 4x1  
0  
0  
0  
0
```

Part(ii)

Calculating inverse using Gauss_Jordan:

```
Ainv=gauss_jordan_inv(A)
```

```
Ainv = 4x4  
-0.250000000000000  0.450000000000000 -0.050000000000000  0.100000000000000  
0 3.200000000000000 -0.800000000000000  0.600000000000000  
0 1.000000000000000 0 0  
0 0.800000000000000 -0.200000000000000  0.400000000000000
```

Verifying that the calculated inverse of A by using gauss_jordan is correct:

```
Ainv*A
```

```
ans = 4x4
    1     0     0     0
    0     1     0     0
    0     0     1     0
    0     0     0     1
```

```
Ainv*A == A*Ainv
```

```
ans = 4x4 logical array
    1     1     1     1
    1     1     1     1
    1     1     1     1
    1     1     1     1
```

Calculating inverse of A using built in function inv:

```
Ainvp = inv(A)
```

```
Ainvp = 4x4
   -0.250000000000000    0.450000000000000   -0.050000000000000    0.100000000000000
         0    3.200000000000000   -0.800000000000000    0.600000000000000
         0    1.000000000000000         0         0
         0    0.800000000000000   -0.200000000000000    0.400000000000000
```

Comparing results by a percent error matrix:

```
pem=zeros(4);
for i=1:4
    for j=1:4
        if Ainvp(i,j)==0
            pem(i,j)=0;
        else
            pem(i,j)=100*abs((Ainv(i,j)-Ainvp(i,j))/Ainvp(i,j));
        end
    end
end
pem
```

```
pem = 4x4
    0     0     0     0
    0     0     0     0
    0     0     0     0
    0     0     0     0
```

The percent error is 0 for every entry.

```
function [x] = gauss_jordan(A,b)
n=size(A,1);
AA=[A b];
for i=1:n
    if (AA(i,i) == 0)
        AA=pivot_row(AA,i);
    end
    AA(i,:)=AA(i,:)/AA(i,i);
    for j=1:n
        if (j ~= i)

```

```

        AA(j,:)=AA(j,:)-AA(j,i)*AA(i,:);
    end
end
end
x=AA(:,n+1);
end

function [A_inv] = gauss_jordan_inv(A)
n=size(A,1);
I=eye(n);
AA=[A I];
for i=1:n
    if (AA(i,i) == 0)
        AA=pivot_row(AA,i);
    end
    AA(i,:)=AA(i,:)/AA(i,i);
    for j=1:n
        if (j ~= i)
            AA(j,:)=AA(j,:)-AA(j,i)*AA(i,:);
        end
    end
end
A_inv=AA(:,n+1:2*n);
end

function [A] = pivot_row(A,i)
[value,index]=max(abs(A(:,i)));
row_temp=A(i,:);
A(i,:)=A(index,:);
A(index,:)=row_temp;
end

```